

2025秋计算机体系结构 大作业

实验基本说明

实验背景

在现代的计算机系统中，硬件结构不断演进，处理器的多级缓存、指令流水线、向量化单元以及多核并行机制使得软件的性能越来越依赖体系结构的相关优化。对于非专业方向的程序员来说，这些实现细节往往被编译器封装成一个“黑盒”，看似无需过多关注；但在实际开发过程中，程序员依旧需要在程序设计的过程中考虑一些相关的设计与优化，以便写出更硬件友好和高效的程序。

在本次实验中，我们将以六种常见的图像处理算法为例，基于 C++ 程序对其进行优化。通过分析算法和硬件特性，探索如何让程序更加充分地利用现代计算机体系结构的特性，从而提升整体的执行性能。

实验运行环境说明

本次课程实验支持使用常见的体系结构和操作系统作为运行环境，同学们可以根据自己的设备选择合适的运行环境。最终我们将使用本地部署的 GitHub Actions 进行自动化测试，因此，请确保你的代码可以在 GitHub Actions 所提供的任何一种软硬件环境上运行，即为：

- x86/64 架构 (SIMD支持 AVX2、SSE 指令集) :
 - Windows Server (g++/clang++/Microsoft Visual C++)
 - Ubuntu (适配 LINUX g++/clang++)
 - macOS (g++/clang++)
- arm64 架构 (SIMD支持 NEON 指令集) :
 - Ubuntu (适配 LINUX g++/clang++)
 - macOS (苹果 M 系列芯片, g++/clang++)

我们将在测试前将编译器更新到最新，因此请放心使用新版本 C++ 特性。请在提交时说明你的代码的运行环境。如果同学们对于运行环境有疑问或使用其他体系结构的运行环境，可以在课程 QQ 群联系助教进行咨询。

项目代码结构

本实验的目录结构如下：

```
final_project/
├── src/
│   ├── image_algorithms.cpp          # 基准实现（不修改）
│   └── image_algorithms_optimize.cpp # 优化文件（修改）
├── include/                          # 头文件
├── benchmarks/                       # Benchmark 测试文件
├── CMakeLists.txt                   # 构建配置
└── README.md                        # 实验文档
```

本实验中，同学们**只需要修改** `image_algorithms_optimize.cpp` 文件以实现算法一定程度上的优化即可，最终需要提交的编写代码相关的文件也只有这个 cpp 文件。

实验内容

算法说明

本实验提供六种图像处理算法的基准实现，同学们需要运用体系结构知识对其进行优化。

1. 高斯滤波

高斯滤波是一种线性平滑滤波，用于图像去噪。本实验使用 3×3 高斯卷积核：

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

对于每个像素 (i, j) ，输出值计算为：

$$\text{output}[i][j] = \sum_{m=-1}^1 \sum_{n=-1}^1 K[m+1][n+1] \times \text{input}[i+m][j+n]$$

性能瓶颈在于大量的内存访问和乘加运算。优化方向包括 SIMD 向量化、缓存优化和循环展开。

2. 幂次变换

幂次变换用于调整图像对比度，变换公式为：

$$\text{output}[i][j] = 255 \times \left(\frac{\text{input}[i][j]}{255} \right)^\gamma$$

本实验中 $\gamma = 0.5$ 。性能瓶颈在于 `std::pow` 函数调用开销。优化方向包括查找表预计算和 SIMD 向量化。

3. Sobel 边缘检测

Sobel 算子通过计算图像梯度来检测边缘。使用两个 3×3 卷积核分别计算水平和垂直方向的梯度：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

梯度幅值为：

$$\text{magnitude} = \sqrt{G_x^2 + G_y^2}$$

优化方向包括 SIMD 向量化和避免平方根运算。

4. 图像转置

矩阵转置操作将图像的行列互换。基准实现按行优先顺序访问，导致大量缓存缺失。优化方向包括缓存分块技术。

5. 均值滤波

均值滤波计算每个像素邻域的平均值。基准实现对每个像素重复计算邻域和。优化方向包括积分图技术和 SIMD 向量化。

6. 图像旋转

将图像顺时针旋转90度。优化方向包括缓存优化和 SIMD 向量化。

运行与测试

本实验使用Google Benchmark框架进行性能测试。为了便于优化，请设置编译优化等级为 `-O0`。

构建步骤：

```
mkdir build && cd build
cmake ..
make
```

运行基准版本（未优化）：

```
./image_benchmarks_baseline
```

修改 `src/image_algorithms_optimize.cpp` 后，重新编译并运行优化版本：

```
make image_benchmarks_optimized
./image_benchmarks_optimized
```

在 `cmake` 配置中还配置了一些其他的测试工具，你可以借助大模型等对于其他的工具进行理解和使用。

实验指导与注意事项

优化技术

在这里我们提供一些常见的优化技术，同学们可以使用其中一种或多种优化技术来实现自己的优化（也可以是这里未提及的优化方法，具体实现由同学们自己决定，理解并实现其中的前两部分内容就可以实现很不错的加速效果）

- 局部性保证**：考虑代码中数据结构的内存布局是否有利于缓存局部性，因为 `vector` 数据结构在内存中可能是不连续的，可以考虑使用连续内存空间来替代。
- 预处理**：某些复杂运算的可能结果可能只有有限的可能，可以考虑在编译时计算出所有可能的结果，在运行时查表即可。
- SIMD 指令加速**：可以考虑使用SIMD指令加速一些复杂的运算，例如高斯滤波中的卷积操作。
 - 对于 x86/64 架构，可以考虑使用 `SSE`、`AVX2`、`AVX512` 等指令集。
 - 对于 arm64 架构，可以考虑使用 `NEON` 指令集。
 - 请注意，SIMD 指令的使用需要考虑数据的对齐问题。
- 缓存优化**：缓存分块（cache blocking/tiling）、内存预取等技术。
- 多线程并行化**：使用 OpenMP 等技术进行并行化。

注意事项

1. 禁止使用编译器优化选项（如 `-O3`），测试环境统一使用 `-O0`
2. 允许使用架构相关选项（如 `-mavx2`、`-march=native`）
3. 多线程测试环境限制为 4 核心
4. 禁止修改算法逻辑，仅允许实现层面的优化
5. 提交前务必验证结果正确性

作业提交要求

一、提交内容

实验报告应包含以下内容：

1. 实验环境说明（硬件配置、操作系统、编译器版本）
2. 性能瓶颈分析（使用性能分析工具的结果）
3. 优化方案设计（针对每个算法的优化思路）
4. 关键代码实现（带详细注释）
5. 性能测试结果（优化前后对比、加速比计算）
6. 结果正确性验证（校验和对比）
7. 总结与反思

二、提交方式

请同学们将实验报告和代码文件按下列要求打包成压缩包，并在超算习堂上进行提交。

- 压缩包命名格式：`[学号]-[姓名].zip`。如 `22300001-张三.zip`
- 提交的压缩包应包含以下两个文件：
 1. **实验报告**：`[学号]-[姓名].pdf`
 - 示例：`22300001-张三.pdf`
 - 包含优化思路、实现过程、优化效果等内容
 2. **优化后的源代码**：`[(avx2|sse|neon)]-[(g++|clang++)]-[学号]-[姓名].cpp`
 - 示例：`avx2-g++-22300001-张三.cpp`
 - 这是你修改后的 `image_algorithms_optimize.cpp` 文件（注意只提交一个 .cpp 代码文件即可）
 - 文件名中包含你使用的指令集和编译器版本

三、评分标准

大作业总分100分，具体评分标准如下：

1. 优化效果（30分）：包括优化前后的性能对比和对结果的分析，其中**绝大多数的分数并不要求优化后的程序性能达到很高的水平**，但是需要有一定的提升，同时需要对优化效果进行合理的解释。

- 如果能做到相对于优化前的程序运行时间有一倍以上的提升（运行时间小于等于原运行时间的50%），即可满足对于优化效果的要求，如能正确解释优化效果，可获得该项目80%以上的分数，此后根据加速效果进行加分（本项参考报告加速比和代码实现，对于近似达到的，会多次测试取平均加速比，将会对分数进行近似处理，也不会扣很多分数）。
 - 如果能做到相对于优化前的程序运行时间有十倍以上的提升（运行时间小于等于原运行时间的10%），且能合理解释优化效果，可获得少量可抵用其他部分分数的额外分数（本项测试以助教评测为准）。
2. 优化思路（30分）：包括对程序的分析、优化思路的合理性、优化方法的选择等。
 3. 代码质量（20分）：包括代码可读性、注释完整性、编程规范等。
 4. 报告质量（20分）：包括结构完整性、表达清晰度、数据充分性等。

四、提交截止时间

详见超算学堂。

附录

常见问题

Q: 如何验证优化后的代码正确性？

A: 使用 `calcChecksum` 函数计算优化前后的校验和，两者应当一致。对于可能产生浮点误差的算法，可以使用 `compareImages` 函数设置容差。

Q: 如何查看详细性能数据？

A: 运行 benchmark 时添加参数：`./image_benchmarks_optimized --benchmark_format=json --benchmark_out=results.json`

Q: 如何限制 OpenMP 线程数？

A: 设置环境变量：`export OMP_NUM_THREADS=4`

Q: 如何检查 CPU 支持的指令集？

A: Linux系统运行：`cat /proc/cpuinfo | grep flags`

Q: 优化版本比基准版本慢怎么办？

A: 可能原因包括 SIMD 指令使用不当、缓存不友好的访问模式、过度的循环展开等。建议使用性能分析工具（如perf、valgrind）定位问题。

参考资料

1. Intel Intrinsics Guide: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>
2. ARM NEON Intrinsics: <https://developer.arm.com/architectures/instruction-sets/intrinsics/>
3. OpenMP Documentation: <https://www.openmp.org/specifications/>
4. Google Benchmark: <https://github.com/google/benchmark>